PANDEMONIUM

WILL CLINGAN

PANDEMONIUM

New User Guide

A sched_ext scheduler for Linux

VERSION 5.24.0

PANDEMONIUM: New User Guide

Copyright © 2026 Will Clingan.

PANDEMONIUM is free software, distributed under the GNU General Public License, version 2. Its source, this guide among it, is at github.com/wllclngn/PANDEMONIUM.

Set in Cardo, designed by David J. Perry. Typeset with XeLaTeX.

First edition, 31 October 2026. Written for PANDEMONIUM v5.24.0.

"What?"

RICHARD M. NIXON

Contents

I	The Price of Distance	1
II	A Circuit for Closeness	3
III	The Shape of the Drum	5
IV	How Long is Too Long	8
V	The Price of the Move	10
VI	The Waterfall	12
VII	The Character of a Task	14
VIII	The Governor, The Rest	16
IX	Reading the Weather	18
X	Every Setting is Measured	21
XI	Where All of This Exists	23
XII	A Road Not Taken	25
XIII	The Hall of Daemons	27

I

The Price of Distance

Somewhere inside the machine in front of you, thousands of times every second, a decision is being made that no one ever sees. There are a handful of processor cores, and there are far more programs that want one than there are cores to give. So something has to choose — this task on that core, for this long, the next one after — and it has to choose faster than thought, continuously, for as long as the computer is on. That something is the scheduler. It is among the oldest and most exhaustively tuned pieces of any operating system, and for most of computing's history it has understood its job as a line at a counter: Who has waited longest? Who was promised the most? Who is next?

pandemonium does not see a line. It sees a graph.

The word needs pinning down, because it does not mean here what it usually means. There is no space in the ordinary sense — no width or height, no coordinates you could point to. The graph is the machine's own anatomy: The cores, and the caches they share. Two cores that share a fast cache sit close together. Two cores in separate clusters sit far apart. And distance, on this graph, measures exactly one thing — what it costs to drag a running task's data from one core to another.

Close is cheap. Far is a cold cache and the time spent refilling it. The whole scheduler is an argument about that single quantity: Where a task belongs, how long it has waited and what it would cost to move it. Almost everything that follows is borrowed. The surprise is where it was borrowed from.

II

A Circuit for Closeness

The idea that a web of connections can be reasoned about begins in 1736, with a man and a map of bridges. Leonhard Euler had been handed an idle puzzle about the Prussian town of Königsberg, whose seven bridges crossed a forking river: Could you walk the town and cross each bridge exactly once, never doubling back? Euler's answer — no — threw away the streets, the river, the distances and kept only what mattered, dots for the landmasses and lines for the bridges. Graph theory was the residue. It has been the mathematics of *what is connected to what* ever since, serenely indifferent to where any of it sits.

The trick pandemonium leans on came a century later, when physicists noticed that a graph and an electrical circuit are the same object in different clothes. Make every connection a resistor, inject a current at one point, read the voltage at another and you have measured a thing called the *effective resistance* between them. Its beauty is that it cannot be fooled by a single good route. It counts every path at once. Two points joined by one thin wire are far apart no matter how short that wire is; two points webbed together by many redundant connections are genuinely close, because current

finds all of them. Klein and Randić gave this a name in 1993 — resistance distance — and an older result, from 1989, proved it a fixed multiple of something almost too tidy to believe: The time a drunkard's random walk would take to wander from one point to the other and stagger back.

So PANDEMONIUM, when it starts, builds the circuit. The cores are the junctions; the shared caches are the resistors, a thick low-resistance wire where a fast cache is shared, a thin one stretched across the width of the chip. It solves the circuit once and learns the true cost of every possible move — not the crude shorthand of *same chip or different chip*, but the real figure, every path through the cache hierarchy folded in. The solving is a Moore-Penrose pseudoinverse of the Laplacian, taken through its eigendecomposition; on a machine small enough to permit it the scheduler instead runs the whole thing in exact integers, by Bareiss elimination, reading each resistance off as a ratio of two determinants, the spanning forests that hold a pair apart over the spanning trees that join everything. Either way the answer reaches the kernel already folded into whole nanoseconds, because the code making decisions there may not touch a floating-point number at all. That number is a task's place in the world. When the task wakes, it goes to the nearest core whose cache is still warm with its data.

III

The Shape of the Drum

In 1787 a German physicist named Ernst Chladni drew a violin bow across the edge of a metal plate strewn with fine sand, and the sand fled the parts of the plate that were shaking and gathered along the curving lines that were not. Each note he coaxed out drew a different figure — the plate's standing waves made visible, the still seams between them traced in sand. Almost two centuries later the mathematician Mark Kac asked the question that haunts the whole field in a single sentence: *can one hear the shape of a drum?* Can the set of tones a thing is able to ring at tell you, by themselves, the shape of the thing? Not always — in 1992 three mathematicians built two differently shaped drums that ring at exactly the same tones — but very nearly: The modes a body can vibrate in are written into its form, its area and the length of its rim among them, and reading the one off recovers most of the other.

A graph can be struck like a plate. The same Laplacian that turned the cache map into a circuit has, hidden inside it, a set of standing waves — its eigenvectors, the graph's own wave functions, the patterns in which it would ring if you could strike it. The lowest of them is flat, the whole machine hum-

ming in unison, and says nothing. The next one up — the fundamental, named the Fiedler vector for the mathematician who in 1973 saw what it was for — has a single nodal line, one seam where the wave changes sign and the sand would gather. And that seam falls where the graph most wants to break in two: The cores on one side genuinely close to one another, the cores on the other side genuinely far. It is the machine's deepest internal boundary, the line a hardware vendor would print on a datasheet as a memory domain — and PANDEMONIUM does not read it off a table the vendor supplies. It reads only which cores share which caches, and hears the boundary itself in the lowest note of the graph.

Then it recurses. Each half of that first cut is a smaller drum with a fundamental of its own, so the scheduler strikes it too and reads the next seam, and strikes each of those pieces in turn, down and down until what remains is a single warm neighborhood that will not split — a leaf. The record of that descent is a tree of domains, the machine's whole structure recovered top to bottom from nothing but its own harmonics. On a small graph the notes can be solved for outright; on a large one that would be ruinous, so the fundamental is instead shaken loose by a power iteration on the normalized Laplacian — the bell is struck and allowed to settle onto its own lowest pitch, the way a real bell finds its tone without anyone computing it. The shape is never declared. It emerges, recursively, from the ringing.

Look closely at any one of those cuts and it has the texture of the whole. The boundary between two domains is not a clean line; it is itself built of smaller boundaries, the sub-drums inside it split by the same rule, and the sub-drums inside those split again — the identical operation at every magnification, which is the defining texture of a fractal. Benoit Mandelbrot found the shape of this in 1967 with the question: *How long is the coast of Britain?* There is no answer. Measure with a shorter ruler and you catch finer inlets, and the length

grows without bound, because the crenellation of the coast repeats at every scale you trouble to look at. PANDEMONIUM's edges are coastlines. Until the descent reaches a leaf there is no magnification at which a domain boundary resolves into a single clean step — zoom in and there is more structure, the same min-conductance seam recurring inside itself. Which is the deep reason none of these edges can be a gate. A gate needs one place to stand the switch, and a self-similar edge offers none; there is only the price, recurring, at every scale at once. The continuity the whole scheduler insists on is not a stylistic choice. It is what an edge with structure at every scale leaves you no alternative to.

And the recursion runs deeper than the tree. This is the same Laplacian that measured distance a moment ago, because distance *is* this spectrum read another way — effective resistance is every one of these standing waves summed together, each weighted by the inverse of its pitch, so the low broad notes dominate and the high fine ones barely register, which is exactly why two well-connected cores come out close. The drunkard's random walk is that same sum a third time. The circuit, the standing wave and the wandering walk are three names for one object, and the scheduler reads all three off it — distance from the whole chord, the domain boundaries from the lowest notes struck recursively, the commute time from the walk. One piece of mathematics, recursing through itself, is the entire sense in which the machine comes to know its own shape.

IV

How Long is Too Long

DISTANCE answers *where*. It says nothing about *when*, and a scheduler that knew only where things belonged would strand them there, starving, with a clear conscience. For *when*, PANDEMONIUM reaches into a different decade and a different necessity.

In the late 2000s the internet developed a quiet illness called bufferbloat. Memory had grown cheap, so the routers and modems between you and everything else were built with enormous buffers, and rather than ever drop a packet they would hoard tremendous backlogs of them — and your video call would stutter, not because the network was full but because your words were sitting in a vast invisible queue, waiting their turn. The cure, in 2012, came from Kathleen Nichols and Van Jacobson, the latter one of the small number of people who had kept the early internet from strangling on its own congestion. They called it Controlled Delay, CoDel — later RFC 8289 — and its insight was a small act of attention. Do not count how many packets are in the queue; that number lies. Measure how long each packet actually *sat* there — its sojourn time. A short queue moving slowly is in

worse trouble than a long queue moving fast, and only the clock can tell you which you have.

A run-queue of waiting tasks is a queue in precisely this sense. PANDEMONIUM notes the instant each task joins a queue and, when it finally leaves to run, subtracts: Now, minus then. That sojourn is the task's reading in time, the literal CoDel metric lifted out of the network stack and laid over the processor, and every queue carries the latest one forward, plus the time since the queue last moved: A bound on how long its oldest waiter has sat. When that climbs past where it ought to be, the scheduler knows — the way CoDel knew a sick router — that something is starving, and that knowing where everything belongs is no longer enough. Its response follows CoDel's own control law, acting sooner each time the queue refuses to drain.

V

The Price of the Move

Now the two measurements converge, and the convergence is the heart of the matter. One core is buried in work; another sits idle. The obvious move is to carry the backlog across. But the move is not free — the carried task arrives on the idle core to find its cache cold, and pays for the relief in misses. So the move has to be priced.

The mathematics of moving load through a network as cheaply as possible has its own distinct origin, in 1955, in a RAND Corporation study of how much traffic the Soviet rail system could bear and where its tightest bottleneck lay. That military question became the maximum-flow problem, formalized a year later by Ford and Fulkerson, and for half a century it lived a life entirely separate from the circuits of resistance distance. Then the two lives folded back together: The fastest known algorithms for maximum flow, the ones that made the field new again in 2011 and culminated in a celebrated almost-linear-time result in 2022, solve the flow by treating the network as — of all the things in the world — an electrical circuit, the same Laplacian arithmetic that measures resistance.

PANDEMONIUM runs that machinery in reverse. It does not solve for the maximum flow; it uses the resistance to *price* the flow. The quantity it computes is called Φ, phi: The graph distance of a proposed move set against the queueing relief that move would buy. A cheap, nearby move happens freely, the instant it helps at all. An expensive move across a cache boundary waits — it does not happen until the backlog it would relieve is genuinely worth the cold cache it would cause. Resistance and flow turn out to be one mathematics seen from two ends, and the scheduler stands at the seam, charging every migration what the distance is worth.

VI

The Waterfall

Pricing a single move is not yet a scheduler; the machine still has to decide, for a core that has just come free, which of all the waiting tasks should run on it — and it answers with a fixed cascade, tried in order, the cheap and the near before the dear and the far. A core looks first at its own small hoard, the tasks already warm in its cache, and nearly always finds its answer there. Only if that is empty does it reach to a near neighbor sharing its fast cache, and even then only if the price is right, the Φ of that steal weighed against the neighbor's backlog. Failing that it draws from the overflow its cache domain shares, oldest first. And only when the whole near neighborhood is spent — its own hoard, its siblings, its local overflow all dry — does a core finally reach across the chip to a far cluster's overflow, and there it pays no price at all, because the cold cache it takes on costs less than a core left idle. A working set is never scattered off the cache that holds it while any nearer work remains. The line between near and far is the one hard boundary in the cascade, and even it is not a gate so much as an order of trying: The dear tier is simply tried last.

Two things sit outside that order, and both are deliberate. A wakee is seated on the nearest idle core the distance walk can claim, and only when the walk finds none is it held at its home rather than queued somewhere colder, freed to move only once its home is congested enough to be worth the cold cache it would land in — Φ pricing turned on a *staying* rather than a move, the near home loosening its grip quickly and the far one holding hard. And beneath the whole cascade runs a safety net: If the oldest task on a core's own queue has sat longer than the machine's own time constant — about thirteen milliseconds on a twelve-thread desktop, longer on a smaller machine and shorter on a larger — the core holding it is preempted for it at once, before any of the pricing, ungated, with no exception able to defer it, and the shared overflow answers to the same kind of floor at a few times that span. It is the one thing the scheduler will not price, and that is exactly why it is the only such thing: A starving task is the single case where the question stops being *what is this worth* and becomes *this runs now*. In a healthy machine it should never fire. It is there so that the pricing everywhere above it is free to be subtle, knowing something absolute is waiting underneath.

VII

The Character of a Task

A VIDEO thread and a disk-flusher want opposite treatment, and serving them alike serves neither. The temptation is to give every task a kind and let the kind decide everything, its slice, its queue, how readily it may shoulder another aside. PANDEMONIUM gave in to it for years, and what is left of it now is small and stated rather than guessed.

What remains comes from what the kernel already knows rather than from a guess. A kernel work queue is marked interactive, because a program is so often found waiting on one, and everything else is marked batch. A thread that declares itself real-time never reaches PANDEMONIUM at all: The kernel's own real-time class sits above sched_ext and runs it first. Each of these is a fact stated outright, not an inference, and a fact cannot quietly stop being true.

For years there was also a scorer that tried to read a task's nature from how often it woke and how long it ran, and the honest account of it is that it never worked. A single field inside it was made to do two incompatible jobs, and the collision meant the score came out zero for every task after its first wake — which is to say the scheduler had been sorting the entire machine into one bucket and calling it a judgment.

Nothing this project ever measured turned out to be a function of it. The scorer is gone, and what replaced it is not a cleverer guess but a ledger: Whether a task used up a full CoDel target on each of its last two turns. A task that did is standing, holding its core rather than passing through it, and only a standing task is given the longer batch slice, never more than four targets of it. That is a record of service rendered rather than a theory of character, and a record cannot be wrong about the past.

The ordering key takes no notion of kind at all — a task's place in line is how long it has waited, oldest first, measured and never guessed.

There is a second ledger, finer than the marks. Every time a task is woken, the scheduler notes who woke it and counts how many times running the same waker has come back. A task woken again and again by one partner is half of a handoff pair, the two ends of a pipe or a client and its server, and belongs beside its partner on a single warm core; when the partner hands it the processor directly, it is seated on the partner's own core. That count steers where a task is placed — but deliberately never whether it is rescued, a line learned the hard way when a steal gated on a task's character once stranded a storm-classified audio thread on a busy core. Placement may consult a task's character. The rescue may not. What must always run must always be reachable, whatever it is.

VIII

The Governor, The Rest

THE TARGET that decides *how long is too long* is not a fixed number. It breathes, and it breathes on a mechanism two and a half centuries old.

Control theory — the science of a system that senses its own output and corrects itself — has a founding machine, James Watt's centrifugal governor of 1788: Two iron balls that spun outward as a steam engine sped up and, spinning, eased the throttle closed, the first contraption to regulate itself with no hand on it. The textbook image of the field is a weight on a spring with friction: The spring pulls it toward rest, the friction keeps it from bouncing forever, then it settles. PANDEMONIUM hangs its starvation target on exactly such a damped oscillator, pulled toward an equilibrium derived from the machine's own shape. Under pressure the target tightens; as things calm it eases back, smoothly, without ringing.

A good regulator should not spend effort every instant, only when the situation has earned it. So when the machine goes quiet, the whole controller parks. It stops recomputing, pins the target at its resting value and costs almost nothing — and the instant a real stall appears it is fully awake again in

the same beat. The scheduler tunes itself, and when there is nothing worth tuning, it rests.

That rest is more careful than it looks. The controller does not simply halt when things go calm; it watches a slowly decaying measure of its own recent effort — decayed on purpose, because the raw measure ripples at twice the system's own frequency, and a controller that woke to that rhythm would feed the very disturbance it meant to damp. As the measure falls it first eases to recomputing every few ticks, then, past a lower mark set well below the first so it cannot dither at the boundary, parks outright: The target pinned to its resting value, the velocity integrator frozen so it cannot quietly wind up and slingshot awake, the arithmetic simply stopped. Parked, it spends a few cheap comparisons a tick watching for the one thing worth waking for — a starvation, a shifted equilibrium, a long-overdue heartbeat — and on any of them wakes fully in the same tick, before the next move is ever priced against a stale target, so that every burst, however the machine came to it, begins from one identical resting state.

It is also why the scheduler keeps no separate alarm for a burst. The structural answer it already holds — the target tightening under pressure, the rescue bounding the tail, a standing task's slice held to a few targets — absorbs a fork storm with no dedicated detector, and the one such detector it once tried was deleted for a pointed reason: It was a single global flag, and a flag raised when the machine got busy became, under a real storm, the very thing the cores tore from one another, the contention it was meant to announce now living in the announcement. A core that reads only its own oldest waiter has nothing it must take from anyone. The controller that learned to rest also learned to listen locally — to keep nothing a crowd could fight over.

IX

Reading the Weather

To SET its own knobs from what the machine is actually doing, PANDEMONIUM borrows from the study of disorder itself. Chaos theory began with Edward Lorenz in the 1960s, watching a rounding error in a weather model bloom into an entirely different forecast: The butterfly effect, the discovery that some systems are exquisitely sensitive to where they begin. Out of that grew a set of instruments for reading the character of any stream of numbers — whether it is steady, chaotic or merely random noise — from nothing but the shape of the data.

The machine hands the scheduler such streams: The share of its decisions that found a core idle and the number of wakeups once a second, and the depth of every run-queue many times a second — each core's own queue and the overflow queue its cache domain shares, sampled four times for every swing of the scheduler's own controller and handed up a second at a time. That is one stream per core and one per domain, not one for the whole machine, which matters more than it sounds. A controller that can see one core starving while another idles, but whose only lever is a single machine-wide setting, observes locally and acts globally: Every local finding is

averaged away at the moment of action. The domain streams are what the settings are derived from, as the next chapter tells. The machine-wide ones go to three instruments, run across a short window every second, each asking the same question from an angle the others cannot reach. Permutation entropy — the measure Christoph Bandt and Bernd Pompe defined in 2002, taken here three samples at a time — reads the wakeups and looks only at the *order* of the samples, the little up-down-up shapes, and throws the magnitudes away; it catches structure that survives any rescaling. The horizontal visibility graph, from Luque and Lacasa in 2009, turns the idle share into a skyline and asks which peaks can see one another over the shorter ones between — random noise and clean rhythm leave different fingerprints in that geometry, where the order-only measure is blind. Recurrence quantification analysis asks of that same idle share how often the system returns to a state it has visited before, and whether those returns fall into long diagonal runs: A truly steady signal revisits its old neighborhoods in streaks, while noise returns by accident and scatters.

There are three and not one because each is fooled by something the others catch. A stream can look random to the first and betray its pattern to the second, or fool both and come apart under the third. So the scheduler asks them different things, and asks each only what its window can hold. Either the order or the skyline may report that the machine has settled into a regime. The recurrences need sixty-four samples before their answer means anything, and the once-a-second window holds sixteen, so there they stay silent rather than guess: A lens without enough data drops out of the verdict instead of vetoing it.

A completely saturated machine and a completely idle one both produce a flat stream, and a flat stream reads as steady to every one of these instruments. So steadiness is not left to them. The scheduler stops retuning only when no queue any-

where held a waiting task in the samples of the last second, on any core or in any domain's overflow queue, and only then do the instruments get their say. Zero is the boundary rather than a tuned threshold: A pegged machine never freezes, and an idle one can.

X

Every Setting is Measured

The scheduler's own settings — how long a slice runs, how eagerly it preempts, how wide a batch job may stretch — have no correct values. The right setting depends entirely on the moment.

There are two ways to meet that. You can *search*: Keep a panel of competing strategies, score them against what went wrong, and let the best one win over time. pandemonium did exactly that for several releases, and then measured it — across eighteen thousand comparisons the searching layer was a coin flip against simply not having it. The trouble is structural rather than a matter of tuning. A searched setting is right several convergence windows later, and only if the machine holds still that long; machines do not hold still.

So each knob is now *derived* instead, from a thing the kernel actually counted a moment ago, and it is derived from the queue a core's cache domain shares, because that is where waiting work collects: A core's own queue is handed its next task almost the moment one arrives, and reads nearly empty even when the machine is buried. How deep that shared queue has been sets how long a slice runs. A deep queue slices shorter so it drains, and a shallow one keeps the ordinary slice,

never a longer one, since a longer slice there only lengthens the wait of whatever arrives next. How bursty the traffic is sets the batch ceiling, measured by the finite-size-corrected statistic Kim and Jo published in 2016 because the naive one lies on short windows. How persistent a backlog has been sets when a rescue fires, read as a Hurst exponent by the wavelet-variance method of Veitch and Abry, because a queue that has been deep on a stubborn domain will still be deep. How much the queues are slowing down — critical slowing, the particular sluggishness a system takes on as it nears a tipping point, which Marten Scheffer's group established as an early warning across ecosystems and climates alike — is measured and reported, and moves nothing: The window it once tightened was one the kernel never read, so the window was removed rather than kept as a dial wired to nothing.

A derived setting is right on the tick it is computed, and it reaches the kernel on that tick, with one courtesy. The kernel runs a controller of its own, a damped oscillator that tightens the scheduler's patience when tasks begin to wait too long. While that controller is already pulling against the pressure, the upper layer holds its new settings back rather than correct the same trouble twice, and sends them once the oscillator has settled. Nothing is learned, and nothing has to converge.

XI

Where All of This Exists

None of it would matter if it could not run where scheduling happens, in the kernel, at the speed of the decisions themselves. The road there runs, once more, through Van Jacobson: In 1992 he and Steven McCanne built the Berkeley Packet Filter, a tiny safe language that let a program filter network packets inside the kernel without the power to crash it. Around 2014 that idea was generalized into eBPF, a sandboxed machine that runs small, verified programs safely in kernel space. And in 2024, sched_ext opened the most jealously guarded room in the building — the CPU scheduler itself — to eBPF, so that for the first time a real, production scheduler could be loaded and run by a stock kernel without anyone recompiling it.

PANDEMONIUM *is* a sched_ext scheduler. Its instant-by-instant work — placing, preempting, sensing a stall — runs as verified code inside the kernel, fast enough to keep pace with thousands of decisions a second and never paying the toll of a trip out to ordinary software. That is the ground the other six ideas stand on.

It runs in two layers that talk to each other, and the division is what makes heavy mathematics affordable. The kernel

layer makes the microsecond decisions and has to be fast, so it does the placing. A second layer, an ordinary program, wakes once a second, reads what the kernel measured across every core and re-derives the knobs from it. Fast reflexes below, arithmetic above.

The division is not arbitrary. Relating one core to another means reading a second core's state, which is precisely what the kernel layer's per-core data structures exist to avoid, and the arithmetic wants real numbers and unbounded loops that verified kernel code cannot have. So the upper layer exists to see the machine whole — the shape the *workload* is making across the cores, which moves every second and which nothing else measures — and the lower layer exists to act inside a microsecond. Neither could do the other's job.

XII

A Road Not Taken

A SYSTEM is also the sum of the elegant ideas it tried and threw away, and three of those discards explain why this one stays calm under load instead of twitching.

The textbook way to detect that *things have just gotten worse* is to accumulate evidence until it crosses a line — the CUSUM chart, out of 1950s industrial statistics. PANDEMONIUM built one and tore it out. On a scheduler an accumulator chatters on the ordinary noise floor; rounding, clock jitter and a single late wakeup all nudge it, so tuned tight it fires at nothing and tuned loose it sleeps through a real storm. The hysteresis already living in the oscillator does the job instead — a deliberate gap between the level that says *tighten* and the level that says *relax*, wider than the jitter, so the system commits to a state and holds it until the world has actually changed.

The reflexive way to tame a noisy measurement is a moving average, and the scheduler refuses it for the signals the chaos instruments read, because smoothing erases the very texture they exist to see. A smoothed line looks calm whether the second was truly steady or merely quiet; the raw window can tell those apart, which is the whole reason the instruments are there.

The last discard is the one that taught the lesson the entire system is built around. On a machine with several cache clusters, in a low-power mode, the scheduler once latched into a self-feeding storm: A real-time task would seize a core, the kernel would hand that core's waiting tasks back to be re-placed across the machine, those placements would displace tasks on other cores, whose own seizures handed back still more — a flood of re-placement that sustained itself and burned the machine in busywork while, from the outside, it looked perfectly idle. The tempting cure was to soften the displacing preempt when things were quiet, and two versions of that cure were built and both removed, because they aimed at the wrong organ. A decisive experiment forced *every* preempt off entirely and the storm did not so much as flinch: The preempt was never the engine, the re-placement flood was. The real fix rate-limits the flood — a core being seized and freed in rapid succession keeps its waiting tasks at home to run the instant it frees, instead of dragging them across the machine and back.

And the shape of that fix is the single law the whole scheduler obeys. Nothing is ever a static on-off gate standing in front of where a task goes. Every decision is a continuous price or a rate, never a hard yes or no. A gate leaks: One branch invites the next, and a special case for every tier after it, and the patch never ends. A price holds, because it does not ask whether to allow the move — it asks what the move is worth, and lets the worth decide.

XIII

The Hall of Daemons

Every running computer is haunted. Out of sight of anything on the screen, dozens of small programs work without rest — listening for connections, flushing data to disk, waking on a timer to sweep up after the others — and they carry a name that is older than the machines and stranger than it looks. They are called daemons. The word arrived by way of a thought experiment: In 1867 the physicist James Clerk Maxwell imagined a tiny, tireless being stationed at a trapdoor between two chambers of gas, sorting the fast molecules from the slow, doing patient invisible work that seemed to cheat the universe. When the programmers around Fernando Corbató built one of the first time-sharing systems at MIT in the early 1960s, they reached for Maxwell's creature to name the processes that labored unseen in the background, and the name held.

It is worth knowing what the Greek actually meant, because it is not what later centuries made of it. A *daimōn* was not a devil. It was a spirit — an attendant, a guardian, something intermediate between the gods and mortals, the inner voice Socrates said warned him away from error. The Christian era darkened the word into the demon, but the older

sense is the one the computer kept: Not an evil spirit but a watchful one, a tireless servant working in the dark on your behalf. At the turn of the 1970s, Ken Thompson and Dennis Ritchie used it in Unix at Bell Labs, and there it became canon — the trailing *d* on a program's name, the silent host of spirits that any living system runs on and no user ever sees.

Which leaves only the question of what to call the place they all gather. For that the name reaches past the physicists and the engineers to a blind poet dictating to his daughters. In 1667 John Milton needed a name for the palace of the fallen angels, raised out of the dark in a single hour, and he minted one from the same Greek: *pan*, all, joined to *daimon* — Pandæmonium, the place of all spirits, the high capital where every fallen angel is summoned to take counsel. The word passed its moorings over the centuries that followed and came to mean what it means in ordinary speech now: Bedlam, uproar, the noise of everything happening at once.

Which is worth holding against what the borrowed ideas are for. A circuit from physics to measure closeness, a congestion metric from the internet to measure waiting, a railway problem run backward to price a move, a steam governor to keep the whole thing steady, three readings of disorder to know the weather, and a kernel sandbox to run it all where it counts. Every one of those is an instrument, and not one was invented here.

What was built here is what they are pointed at. Other schedulers are handed the machine's shape in a table the manufacturer wrote; this one strikes the graph and reads the shape out of the ringing, so the boundary it respects is the boundary that is actually there rather than the one a datasheet claims. Other schedulers gate a measurement off until it is trustworthy and then take it at full strength; this one prices confidence continuously, so a reading from a thin window nudges and a reading from a complete one swings, and nothing arrives all at once. And CoDel was built to decide what a router should

DROP; here its clock decides what runs next, which is not what it was for and is the whole reason this scheduler behaves the way it does.

Beneath all of it, one conviction: That a machine should not be told its own shape and handed a list of rules, but should work the shape out for itself and charge an honest price for every move across it. A name that has come to mean everything happening at once, attached to the one in the room that keeps the steadiest time.